

DriveEase

Services Layer

Business logic: rules, validation and orchestration

ASP.NET Core 9.0 MVC | Entity Framework Core | PostgreSQL (Supabase)

What the Services layer does

Services hold the **business logic** - the rules that make DriveEase more than a database editor. Each feature has an **interface** (I*Service) and an **implementation**. Controllers depend on the interface, and the DI container (configured in Program.cs) injects the implementation. All services are registered as **Scoped** and operate on the singleton `IDataStore`, using `lock` blocks for thread safety.

A common return pattern

Most write operations return a tuple (`bool Success, string Message, ...`). This lets a controller check success, and on failure push **Message** straight into `ModelState` to redisplay the form with a helpful error - no exceptions for expected validation failures.

The six services

Service	Interface	Responsibility
UserService	<code>IUserService</code>	Registration, login, account status.
VehicleService	<code>IVehicleService</code>	Listing, search/filter, recommendations, fleet admin.
BookingService	<code>IBookingService</code>	Availability, pricing, create/cancel bookings.
PaymentService	<code>IPaymentService</code>	Process payment, confirm booking.
ReviewService	<code>IReviewService</code>	Add reviews, recompute vehicle rating.
AdminService	<code>IAdminService</code>	Dashboard aggregate statistics.

1. UserService

Method	Returns	Business rules
RegisterAsync	(ok, msg, User?)	Rejects duplicate email; rejects duplicate mobile; creates a Customer, IsActive=true. Password stored as plain text (demo).
LoginAsync	(ok, msg, User?)	Finds user by email OR mobile; rejects if not found, if inactive, or if password mismatch. Generic message avoids leaking which field was wrong.
GetUserByIdAsync	User?	Lookup by id.
GetAllUsersAsync	List	All users (admin use).
ToggleUserStatusAsync	bool	Flips IsActive - soft enable/disable.

2. VehicleService

Method	Returns	Business rules
GetAllVehiclesAsync	List	Excludes UnderMaintenance vehicles.
SearchAndFilterAsync	List	Filters by location, type, price range, fuel, transmission; sorts by price or rating.
GetVehicleByIdAsync	Vehicle?	Lookup by id.
GetRecommendationsAsync	List	Rule-based scoring engine (see below).
AddVehicleAsync	(ok, msg)	Rejects duplicate license plate.
UpdateStatusAsync	bool	Change Available/Booked/UnderMaintenance.

Recommendation scoring (out of ~100)

Factor	Points	Logic
Seating fit	30 / 20	30 if capacity == passengers (or +1), else 20; skip if too small.
Budget fit	30 / 15	30 if within budget, 15 if <=120%, skip if well over.
Preferred type	25 / 15	25 on exact match, 15 if no preference given.
Rating	up to 15	AverageRating x 3.

Only **Available** vehicles are considered; results are sorted by score descending, each with a human-readable Reason.

3. BookingService

Method	Returns	Business rules
IsVehicleAvailableAsync	bool	False if vehicle missing or under maintenance; else checks date overlap against non-cancelled bookings using: pickup < existing.Return AND return > existing.Pickup.
CalculatePriceAsync	(ok, msg, Price?)	Validates dates; duration = ceiling of hours/days (min 1); applies weekend surge multiplier; validates & applies promo (active, in date window, min amount; discount capped at base).
CreateBookingAsync	(ok, msg, Booking?)	Re-checks availability + price, then creates a booking in Pending status with a generated reference DE-yyyyMM-####.
GetBookingByIdAsync	Booking?	Lookup by id.
GetUserBookingsAsync	List	A user's bookings, newest first.
CancelBookingAsync	(ok, msg)	Owner-only; cannot cancel a Completed or already-Cancelled booking; sets status to Cancelled.

Pricing formula

```
duration = ceil( (return - pickup) in hours or days ), min 1
subtotal = duration * (HourlyRate or DailyRate)
basePrice = round( subtotal * surgeMultiplier, 2 )
discount = percentage-of-base OR flat amount (capped at basePrice)
total = max(0, basePrice - discount)
```

4. PaymentService

Method	Returns	Business rules
ProcessPaymentAsync	(ok, msg, Payment?)	Booking must exist and be Pending; amount must equal TotalAmount (full payment only). On success creates a Successful Payment, links it, and flips the booking to Confirmed.
GetPaymentByBookingIdAsync	Payment?	Fetch a booking's payment.

5. ReviewService

Method	Returns	Business rules
AddReviewAsync	(ok, msg)	Reviewer must own the booking; booking must be Completed; no duplicate review per booking. After saving, recomputes the vehicle's ReviewCount and AverageRating.
GetVehicleReviewsAsync	List	A vehicle's reviews, newest first.

6. AdminService

Method	Returns	What it aggregates
GetDashboardStatsAsync	AdminDashboardViewModel	Customer count; vehicle counts by status; booking counts by status; total revenue from successful payments; the 5 most recent bookings.

Cross-cutting notes

Async signatures, sync body. Methods return Task but run synchronously over in-memory lists (Task.FromResult). This keeps the public contract async-ready so that swapping in EF Core (real awaited DB calls) later won't change callers.

Thread safety. Because the store is a shared singleton, services lock the relevant list during reads/writes. Once EF Core (scoped DbContext) becomes the source of truth, this locking becomes unnecessary.

Validation split. Simple field checks live on the ViewModels (DataAnnotations); cross-record rules that a form cannot know (duplicate email, date overlap, promo validity, ownership) live here in the services.